

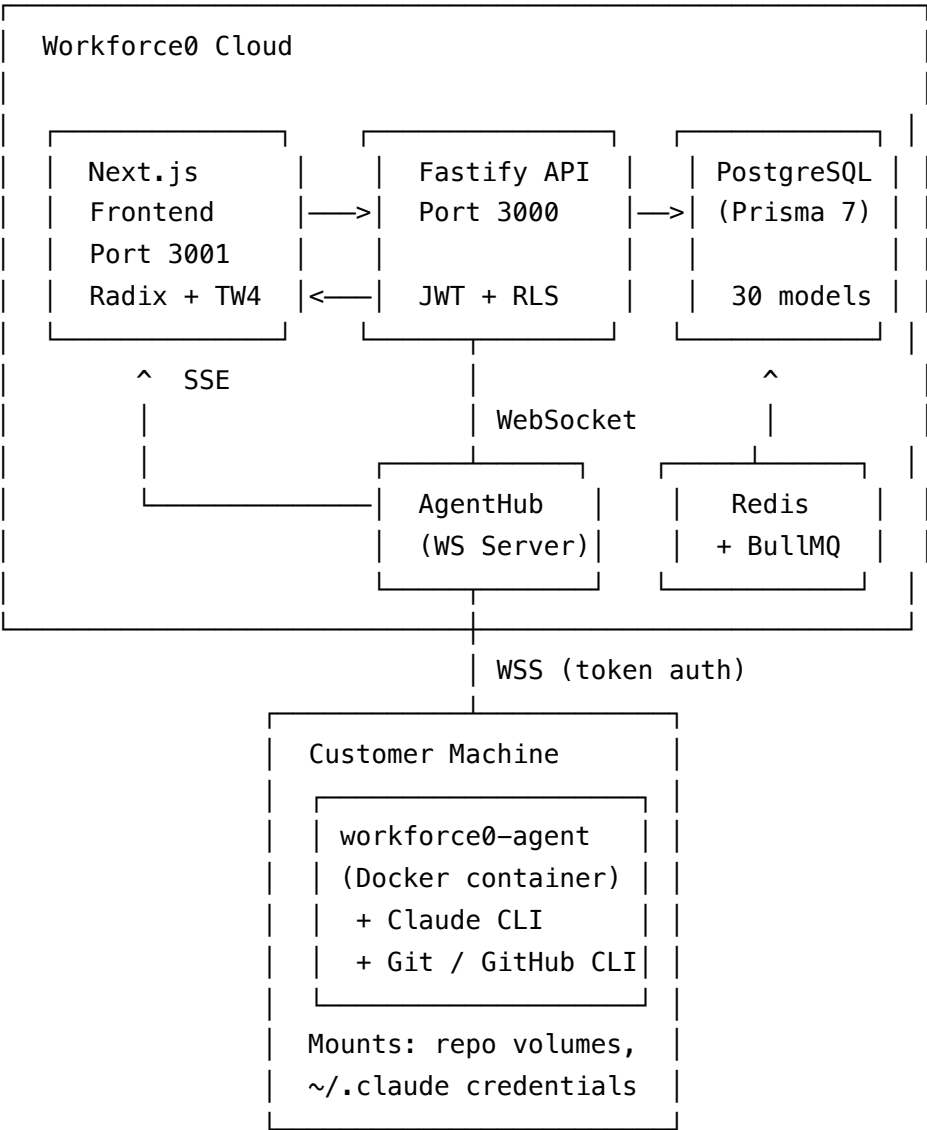
Workforce0 Technical Architecture

Workforce0 – Technical Architecture Guide

For CTOs, Lead Developers, and Engineering Leaders

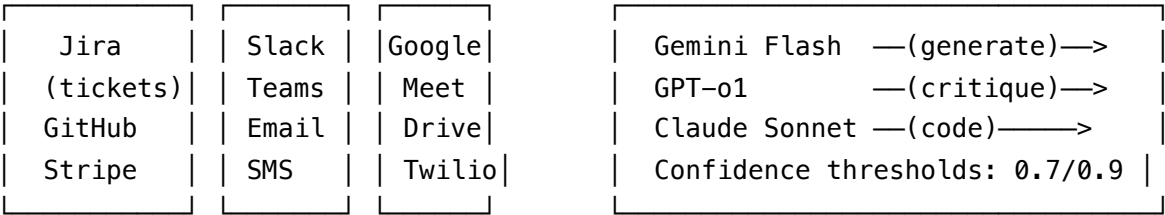
System Overview

Workforce0 is an AI Workforce Platform that transforms meeting transcripts into deployed software and business actions. AI agents (BA, Dev, QA) do the work; humans approve. The system uses multi-model consensus (Gemini generates, GPT-o1 critiques, Claude codes) and a self-improving skill system that learns from outcomes.

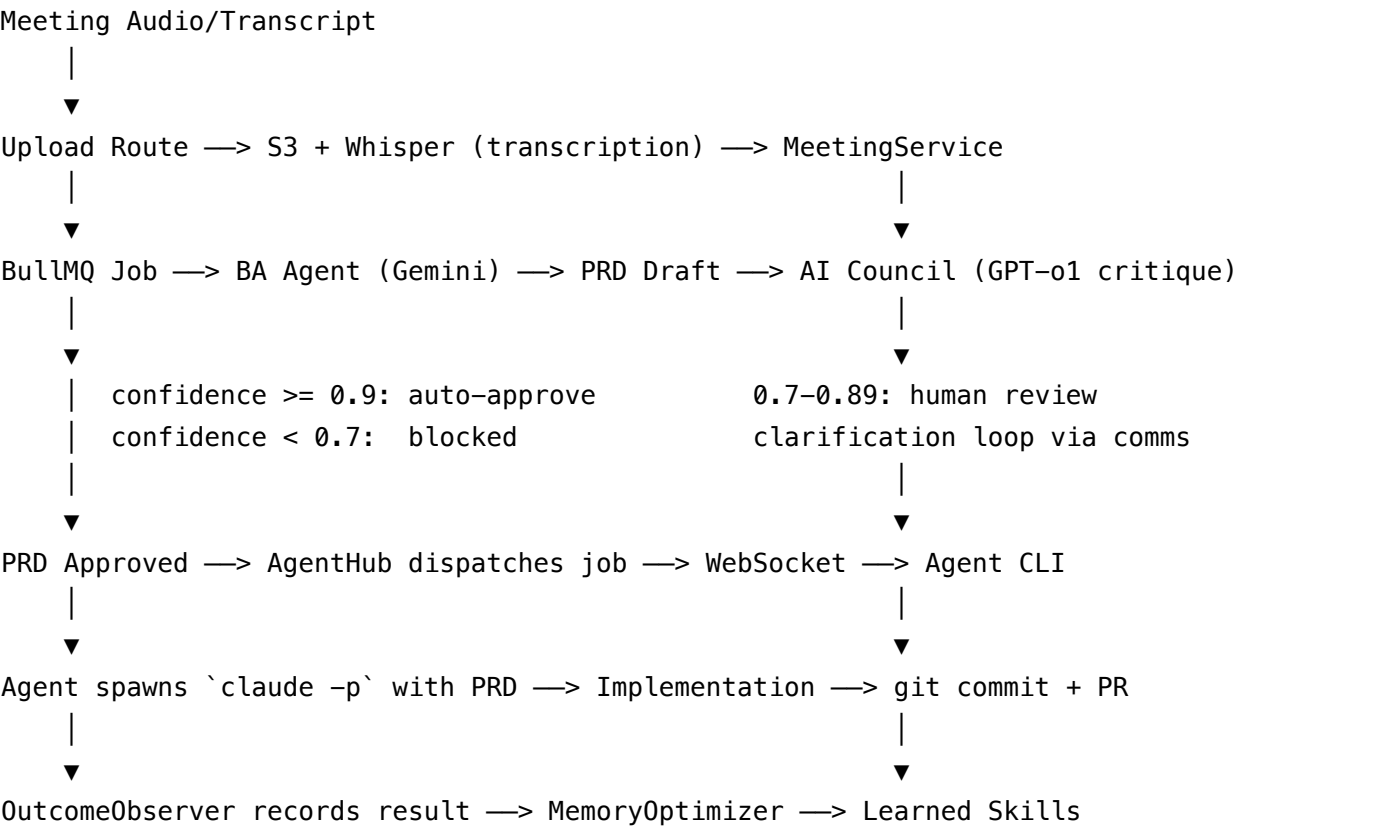


External Integrations:

AI Pipeline:



Request Flow



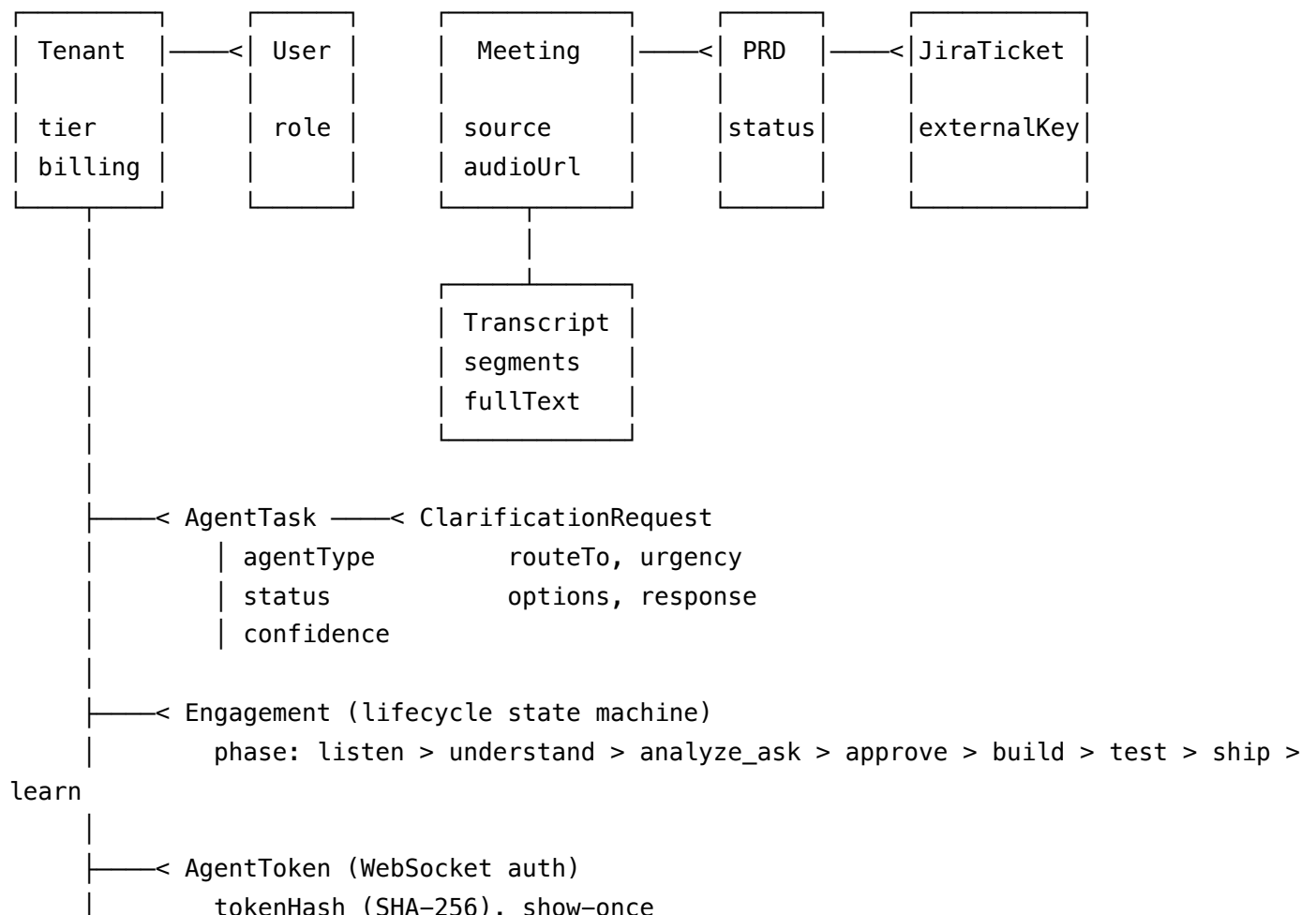
Tech Stack

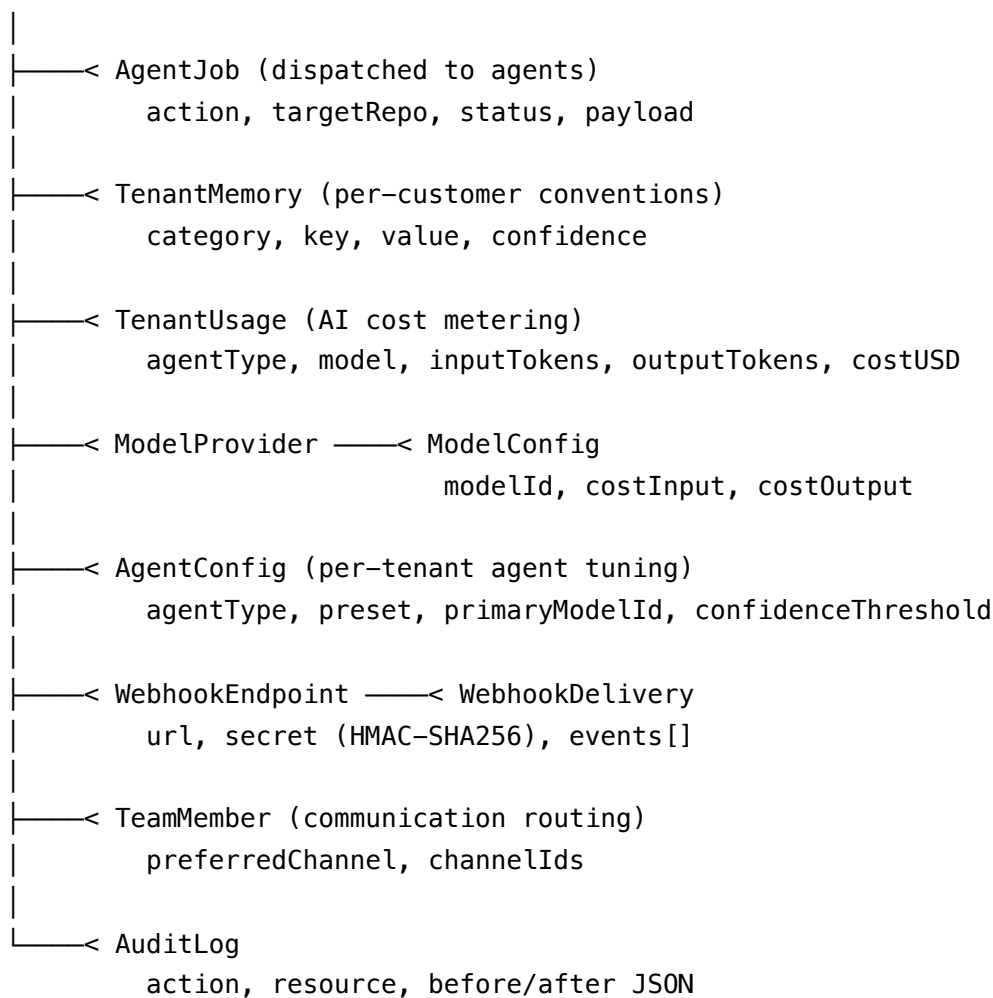
Layer	Technology	Purpose
Frontend	Next.js, React, Radix UI, Tailwind CSS v4	Executive-friendly UI
Backend	Fastify, TypeScript, Node.js 20	API server with plugin architecture
Database	PostgreSQL 15 + Prisma 7 (adapter pattern)	Data persistence, multi-tenant RLS
Cache/Queue	Redis 7 + BullMQ	Caching, job processing (10 job types)
AI (BA Agent)	Google Gemini 2.0 Flash	Meeting analysis, PRD generation
AI (Critique)	OpenAI GPT-o1	Adversarial PRD review

Layer	Technology	Purpose
AI (Dev Agent)	Anthropic Claude Sonnet 4 (user's subscription)	Code generation (runs on user's machine)
AI (Voice)	OpenAI Realtime API + Twilio	Dial-in voice bot for meetings
Real-time	WebSocket (ws) + SSE	Agent connections, UI event streaming
Agent Runtime	Node.js CLI / Docker (node:20-alpine)	Remote code execution
Auth	JWT (httpOnly cookies) + WorkOS SSO	Authentication, enterprise SSO
Observability	OpenTelemetry, Pino structured logging	Distributed tracing, log aggregation
Communication	Slack, Email (SendGrid), Teams, WhatsApp, SMS	Multi-channel notifications
Billing	Stripe	Subscription management
Storage	AWS S3	Meeting audio uploads

Data Model

30 Prisma models organized by domain. Key models and their relationships:





Global models (not tenant-scoped):

- **LearnedSkill** – AI-generated skills from outcome analysis (candidate/active/demoted lifecycle)
- **AgentOutcome** – Records of agent task results for the learning loop
- **MeetingInsights** – AI-generated meeting summaries, action items, decisions
- **PRDTemplate** – Configurable PRD section templates (system-default or tenant-specific)

Multi-Tenancy Implementation

Tenant isolation is enforced at three levels:

1. **Prisma Client Extensions** – `createTenantAwarePrisma()` wraps the Prisma client with `$extends` to automatically filter all queries on tenant-scoped models by `tenantId`.
2. **Middleware** – `registerTenantMiddleware()` extracts `tenantId` from JWT and attaches it to request context. In development, the `X-Tenant-ID` header is also accepted.
3. **Route-level** – All `/api/*` routes require JWT authentication, and `tenantId` is extracted from the token payload.

Agent Architecture

AgentHub – WebSocket Server

The AgentHub manages persistent WebSocket connections between the Workforce0 cloud and customer-deployed agents. It handles authentication, job dispatch, progress tracking, and result collection.

Connection limits: - Max 20 agents per tenant - Max 500 agents total - Max payload: 5 MB - Auth deadline: 10 seconds - Heartbeat timeout: 90 seconds

WebSocket Protocol

All messages are JSON. The protocol uses a strict lifecycle: auth, register, then job dispatch.

Agent to Hub:

// 1. Authenticate with token

```
{ type: 'auth', token: string }
```

// 2. Register capabilities

```
{ type: 'register', repos: string[], capabilities: string[] }
```

// 3. Heartbeat

```
{ type: 'ping' }
```

// 4. Acknowledge job receipt

```
{ type: 'job_ack', jobId: string }
```

// 5. Report progress

```
{ type: 'job_progress', jobId: string, message: string, percent: number, logs?: string[] }
```

// 6. Report result

```
{ type: 'job_result', jobId: string, status: 'done' | 'failed', data: Record<string, unknown> }
```

Hub to Agent:

// Auth response

```
{ type: 'auth_ok', agentId: string, tenantId: string }
```

```
{ type: 'auth_error', message: string }
```

// Registration acknowledged

```
{ type: 'registered', repos: number }
```

// Heartbeat response

```
{ type: 'pong' }
```

// Job dispatch

```
{ type: 'job', jobId: string, action: string, payload: Record<string, unknown> }
```

```
// Job completion acknowledgment
{ type: 'job_complete', jobId: string }

// Resume stalled job
{ type: 'job_resume', jobId: string, payload: Record<string, unknown> }

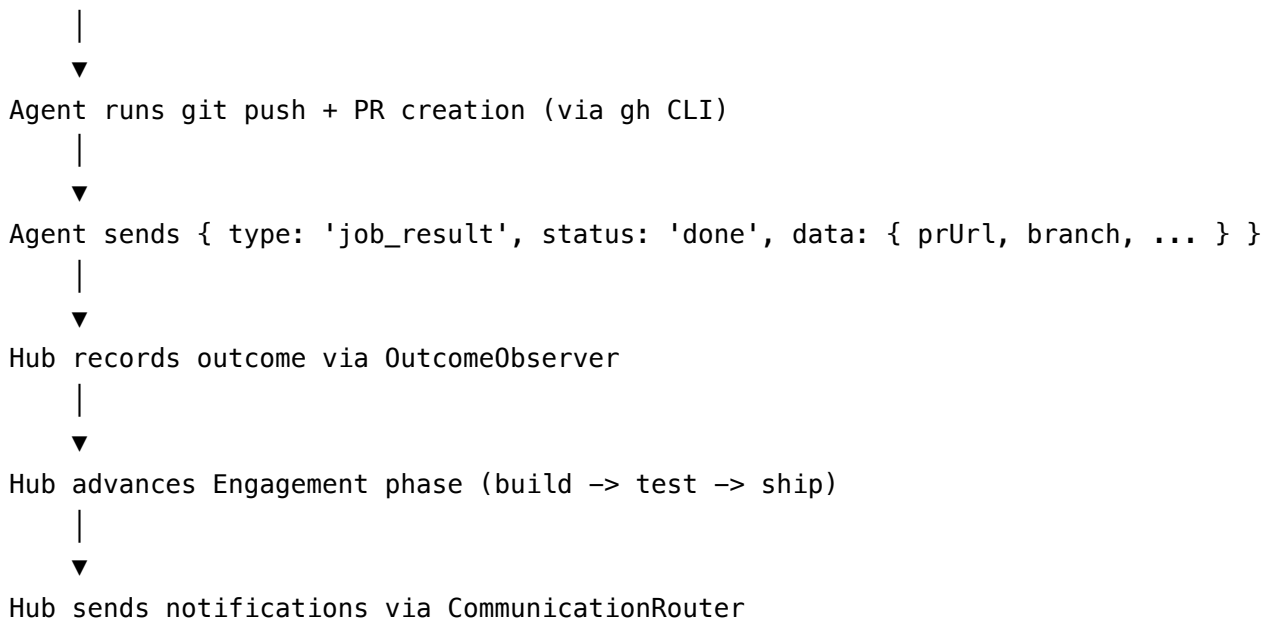
// Graceful shutdown notice
{ type: 'server_shutdown', reconnectAfter: number }
```

Agent Connection State

```
interface AgentConnection {
  agentId: string;
  tenantId: string;
  ws: WebSocket;
  repos: string[];           // e.g. ["acme/backend", "acme/frontend"]
  capabilities: string[];    // e.g. ["claude", "git", "npm", "gh"]
  activeJobs: Set<string>;
  maxActiveJobs: number;    // Default: 3
  connectedAt: Date;
  lastPingAt: Date;
  authenticated: boolean;
}
```

Job Lifecycle

```
PRD Approved (UI or AI Council auto-approve)
|
▼
AgentJobQueue.enqueue({ action: 'implement_prd', targetRepo, payload: { prdContent,
branch } })
|
▼
AgentHub finds idle agent matching targetRepo
|
▼
Hub sends { type: 'job', ... } to agent WebSocket
|
▼
Agent sends { type: 'job_ack', jobId }                (5-min ack deadline)
|
▼
Agent spawns `claude -p --output-format json --allowedTools
Edit,Write,Bash,Read,Grep,Glob`
|   └─ PRD content piped via stdin
|
▼
Agent sends periodic { type: 'job_progress', percent, message }
|   └─ Progress patterns: reading (5%) -> analyzing (15%) -> writing code (30-45%)
|                        -> tests (60%) -> committing (80%) -> pushing (90%) -> done (95%)
```



Agent CLI Usage

Direct invocation

```
workforce0-agent \
  --token wf0_abc123 \
  --repos acme/backend:/path/to/repo,acme/frontend:/path/to/frontend \
  --server wss://api.workforce0.dev/agent/ws \
  --max-jobs 3 \
  --job-timeout 45 \
  --require-approval \
  --verbose
```

Docker (recommended)

```
docker run -d --name workforce0-agent \
  -e WF0_TOKEN=wf0_xxx \
  -e WF0_SERVER=wss://api.workforce0.dev/agent/ws \
  -v ~/.claude:/home/agent/.claude \
  -v ~/code/myapp:/workspace/myapp \
  workforce0/agent --repos acme/app:/workspace/myapp
```

Token resolution order: --token flag > WF0_TOKEN env var > ~/.workforce0/token file

Security Model

- **Token auth:** SHA-256 hashed, show-once token generated via UI. Stored as tokenHash in AgentToken table, never in plaintext.
- **Docker isolation:** Agent runs as non-root user (uid 1001) inside a minimal Alpine container. Repo directories are mounted as volumes – the agent cannot access the host filesystem beyond mounted paths.
- **--require-approval flag:** When set, the agent displays job details in the terminal and waits for explicit y/N confirmation before executing. In non-TTY mode (e.g., CI), auto-approves.

- **No credentials on Workforce0 servers:** Claude API keys (user's subscription) and repo access stay on the user's machine. The server only sends PRD content and receives results.
 - **Tool allowlist:** Claude CLI is invoked with `--allowedTools Edit,Write,Bash,Read,Grep,Glob` – no network access, no arbitrary tool execution.
-

AI Pipeline

BA Agent (Server-side)

The BA Agent processes meeting transcripts to generate Product Requirements Documents.

- **Model:** Google Gemini 2.0 Flash (configurable per tenant via ModelRegistry)
- **Architecture:** BaseConsultant -> AgentLoop -> ModelClient
- **Skill injection:** Foundation skills + learned skills injected into system prompt
- **Token budget:** 1,500 tokens max for skill injection (approx. 6,000 characters)
- **Output:** Structured PRD with objectives, requirements, acceptance criteria, risks, architecture options

AI Council (Multi-Model Consensus)

```
interface CouncilDecision {  
  decision: 'approved' | 'needs_revision' | 'rejected' | 'needs_human_review';  
  confidence: number;  
  prd: GeneratedPRD;  
  critique?: CritiqueResult;  
  votes: ConsensusVote[];  
  consensusReached?: boolean;  
}
```

Model roles by agent type:

Agent	Primary (Generate)	Critique/Review
BA Agent	Gemini Flash	GPT-o1 (adversarial)
Dev Agent	Claude Sonnet 4	Gemini + GPT-o1 (triple review)
Sales Agent	Gemini Flash	GPT-o1
Marketing Agent	Gemini Flash	GPT-o1

Confidence thresholds: - ≥ 0.9 : Auto-approved, no human review needed - $0.7 - 0.89$: Mandatory human review, clarification loop triggered - < 0.7 : Blocked, escalated to stakeholder
- Max 2 revision loops before escalating to human regardless of confidence

Fallback chains: If a model provider is unavailable, the council degrades gracefully. The AI Council constructor accepts nullable services – when OpenAI is not configured, single-model mode is used with Gemini only.

Dev Agent (Client-side)

- **Model:** Anthropic Claude Sonnet 4 (user's own subscription)
- **Execution:** Runs on user's machine via Docker container
- **Process:** Agent spawns `claude -p` with PRD piped via stdin
- **Actions:** `implement_prd`, `review_pr` (extensible)
- **Output:** Git commits + pull request (via GitHub CLI)

Prompt Construction

The JobExecutor builds action-specific prompts:

```
// implement_prd action
`You are an expert software engineer. Your task is to implement the following PRD.
Target branch: ${branch}

## PRD
${prdContent}

## Instructions
1. Read the existing codebase to understand its structure and conventions.
2. Implement all features described in the PRD.
3. Create a git branch named "${branch}" if it does not already exist.
4. Commit all your changes with a descriptive commit message.

IMPORTANT: Do NOT push the branch or create a pull request -- just implement, commit, and
stop.`
```

The agent handles git push and PR creation separately after Claude finishes.

Skill System (Self-Improving)

Architecture

```
BaseConsultant
|
├─ consultantConfig.rolePrompt      (agent-specific system prompt)
├─ consultantConfig.roleSkillTargets (which skills to load: ['all', 'ba'])
|
└─ SkillLoader.resolveAllSkills(targets)
    |
    └─ Foundation Skills (13 total, immutable, checked into code)
        └─ all/ : security, coding-standards, quality-gates
```

```

|       |— ba/      : api-design, database-migrations, prd-patterns
|       |— dev/     : tdd-workflow, security-review, backend-patterns, docker-
patterns
|       |— qa/      : e2e-testing, test-coverage, security-scan
|
|— Learned Skills (from DB, cached in Redis, max 5 active)
    |— Loaded from LearnedSkill table where status='active'
        Cached for 30 minutes (LEARNED_SKILLS_CACHE_TTL = 1800s)

```

Skill Types

```

type SkillTarget = 'all' | 'ba' | 'dev' | 'qa' | 'meeting_brain' | 'supervisor';
type SkillScope = 'foundation' | 'learned';
type LearnedSkillStatus = 'candidate' | 'active' | 'demoted';

```

```

interface FoundationSkill {
  name: string;           // e.g. "TDD Workflow"
  version: string;        // e.g. "1.0.0"
  target: SkillTarget;
  content: string;        // Markdown injected into system prompt
}

```

```

interface LearnedSkill {
  id: string;
  name: string;
  content: string;
  target: SkillTarget;
  confidence: number;     // 0.0 - 1.0
  sourceOutcomes: number; // How many outcomes contributed
  positiveRate: number | null;
  negativeRate: number | null;
  status: LearnedSkillStatus;
}

```

System Prompt Assembly

BaseConsultant.buildSystemPrompt() constructs the prompt in layers:

[Role Prompt]	<- Agent-specific instructions
[## Firm Methodology]	<- Foundation skills matching target
[## Institutional Knowledge]	<- Learned skills (token-budgeted)

Token budget enforcement: Foundation skills fill first, learned skills use remaining budget (max 1,500 tokens total for skills, at 4 chars/token estimate).

Learning Loop

Agent completes task



```
OutcomeObserver.record({ agentType, taskId, result, confidenceScore, toolsUsed, stepCount
})
|
▼
MemoryOptimizer (BullMQ job, Gemini-powered)
|   Analyzes patterns across outcomes
|   Generates skill candidates
|
▼
LearnedSkill created with status='candidate', confidence=0.3
|
▼
Promotion criteria: confidence >= threshold, positive outcome rate
|
▼
status='active' (max 5 active skills per target)
```

Bloat Prevention

- **Keyword overlap check:** Skills with >60% keyword overlap are considered redundant and demoted
 - **Hard cap:** Maximum 5 active learned skills per target type
 - **Token budget:** 1,500 tokens max for all skill injection (foundation + learned)
 - **Admin kill switch:** /api/admin/learned-skills routes allow manual demotion/deletion
-

Dependency Injection

Services are registered on the Fastify instance via the decorator pattern. After `setupDependencies()`, all services are available via `app.services.*`.

Service Registry

```
interface Services {
  // Infrastructure
  prisma: PrismaClient;           // With RLS extensions
  redis: Redis;                   // General caching

  // Data Access
  meetingRepository: MeetingRepository;
  taskRepository: TaskRepository;
  prdRepository: PRDRepository;

  // Core Business Logic
  meetingService: MeetingService;
  baAgentService: BAAgentService;
  engagementService: EngagementService;
  queueService: QueueService;
```

```

// AI
geminiService: GeminiService;
openaiService: OpenAIService;
aiCouncil: AICouncil;

// Integrations (nullable = gracefully disabled when unconfigured)
jiraService: JiraService;
googleChatService: GoogleChatService;
googleDocsService: GoogleDocsService;
githubService: GitHubService | null;
twilioVoiceService: TwilioVoiceService | null;
googleOAuthService: GoogleOAuthService | null;

// Communication
communicationRouter: CommunicationRouter; // Routes to Slack, Email, Teams, WhatsApp,
SMS

// Metering & Audit
usageService: UsageService; // AI token & cost tracking
auditService: AuditService; // Action audit log
webhookService: WebhookService; // Outgoing webhook delivery

// Real-time
sseService: SSEService; // Server-Sent Events for UI
agentHub: AgentHub; // WebSocket agent connections
agentJobQueue: AgentJobQueue; // Job queue for agent dispatch

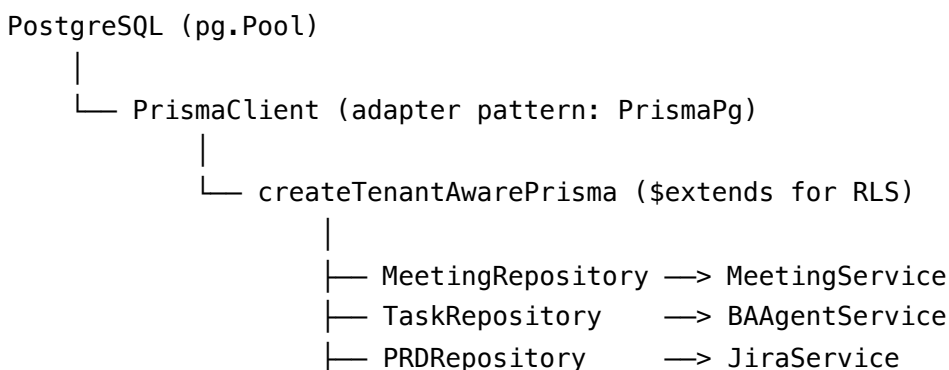
// Auth
workosService: WorkOSService | null; // Enterprise SSO

// Storage & Transcription
s3Service: S3Service | null;
transcriptionService: TranscriptionService | null; // OpenAI Whisper

// Meeting
transcriptBuffer: TranscriptBufferService; // Batches real-time chunks
clarificationTimeoutService: ClarificationTimeoutService;
}

```

Dependency Graph



- |— EngagementService
- |— MemoryService
- |— UsageService
- |— AuditService
- |— AgentJobQueue —> AgentHub

Redis (ioredis)

- |
- |— General caching (main connection)
 - |— MemoryService, SkillLoader (learned skill cache)
- |
- |— BullMQ (separate connection, maxRetriesPerRequest: null)
 - |— QueueService
 - |— 10 Job Types:
 - MEETING_PROCESS, BA_AGENT_PROCESS, JIRA_SYNC,
 - NOTIFICATION, DEV_AGENT_PROCESS, QA_AGENT_PROCESS,
 - CLARIFICATION_TIMEOUT, CLARIFICATION_REMINDER,
 - MEMORY_OPTIMIZER, MEETING_TRANSCRIBE

External APIs

- |
- |— GeminiService —> AICouncil (primary generator)
- |— OpenAIService —> AICouncil (critique model)
- |— JiraService (ticket creation)
- |— GoogleChatService (notifications)
- |— GoogleDocsService (PRD export)
- |— GitHubService (PR management)
- |— TwilioVoiceService (dial-in bot)
- |— SlackChannel, EmailChannel, TeamsChannel, WhatsAppChannel, SMSChannel
- |— S3Service (audio storage)
- |— TranscriptionService (Whisper API)
- |— GoogleOAuthService (Meet/Drive integration)
- |— WorkOSService (enterprise SSO)
- |— Stripe (billing webhooks)

Graceful Degradation

All optional services disable cleanly when their environment variables are not set:

```
// Pattern used throughout di-container.ts:
const githubService = config.GITHUB_TOKEN
  ? new GitHubService({ token: config.GITHUB_TOKEN, ... })
  : null;

if (!githubService) {
  logger.warn('GitHubService disabled (no GITHUB_TOKEN configured)');
}
```

Services that follow this pattern: TwilioVoiceService, GitHubService, Google0AuthService, S3Service, TranscriptionService, Work0SService, MemoryOptimizerAgent. When null, dependent code skips the integration rather than throwing.

API Reference

Authentication

Method	Path	Description
POST	/api/auth/signup	Create tenant + admin user
POST	/api/auth/login	Get JWT access + refresh tokens
POST	/api/auth/refresh	Refresh access token

JWT tokens are delivered as httpOnly cookies (wf0_access) with fallback to Authorization: Bearer header.

Meetings

Method	Path	Description
GET	/api/meetings	List meetings (cursor paginated)
GET	/api/meetings/:id	Get meeting with transcript
POST	/api/meetings	Schedule meeting bot
DELETE	/api/meetings/:id	Cancel meeting
GET	/api/meetings/:id/transcript	Get transcript segments

Meeting Upload & Transcription

Method	Path	Description
POST	/api/meetings/upload/transcript	Upload transcript text
POST	/api/meetings/upload/audio	Upload audio (S3 + Whisper)

Agents & PRDs

Method	Path	Description
POST	/api/agents/ba/process	Trigger BA Agent on meeting
GET	/api/agents/tasks	List agent tasks
GET	/api/agents/tasks/:id	Get task details
GET	/api/agents/prds	List PRDs
GET	/api/agents/prds/:id	Get PRD

Method	Path	Description
POST	/api/agents/prds/:id/approve	Approve PRD
POST	/api/agents/prds/:id/reject	Reject PRD
POST	/api/agents/prds/:id/tickets	Generate Jira tickets
GET	/api/agents/prds/:id/download	Download PRD as document
POST	/api/agents/clarifications/:id/respond	Answer clarification

Agent Hub (Token Management & Job Status)

Method	Path	Description
POST	/api/agents/tokens	Create agent token (returned once)
GET	/api/agents/tokens	List tokens (hints only)
DELETE	/api/agents/tokens/:id	Revoke token
GET	/api/agents/status	Connected agents status
GET	/api/agents/jobs	List agent jobs

Engagements

Method	Path	Description
GET	/api/engagements	List engagements
GET	/api/engagements/:id	Get engagement details
POST	/api/engagements/:id/advance	Advance lifecycle phase
POST	/api/engagements/:id/pause	Pause engagement
POST	/api/engagements/:id/resume	Resume engagement

Model Configuration

Method	Path	Description
GET	/api/models/config	Get tenant model configuration
PUT	/api/models/config	Update model assignments per agent
GET	/api/models/available	List available models

Team & Communication

Method	Path	Description
GET	/api/team	List team members
POST	/api/team	Add team member
PUT	/api/team/:id	Update member (channel, role)

Method	Path	Description
DELETE	/api/team/:id	Remove team member

Settings & Dashboard

Method	Path	Description
GET	/api/settings	Get tenant settings
PUT	/api/settings	Update tenant settings
GET	/api/dashboard	Dashboard aggregated data

Notifications & Events

Method	Path	Description
GET	/api/notifications	List in-app notifications
GET	/api/events/stream	SSE event stream

Webhooks (Outgoing)

Method	Path	Description
GET	/api/webhooks	List webhook endpoints
POST	/api/webhooks	Create webhook endpoint
PUT	/api/webhooks/:id	Update endpoint
DELETE	/api/webhooks/:id	Delete endpoint

Audit Log

Method	Path	Description
GET	/api/audit-log	Query audit events

Admin

Method	Path	Description
GET	/api/admin/learned-skills	List learned skills
PATCH	/api/admin/learned-skills/:id	Demote/activate skill
DELETE	/api/admin/learned-skills/:id	Delete skill

Webhooks (Incoming, no auth – signature verified)

Method	Path	Description
POST	/webhooks/jira	Jira event callbacks

Method	Path	Description
POST	/webhooks/github	GitHub event callbacks
POST	/webhooks/gchat	Google Chat interaction
POST	/webhooks/google-drive	Drive change notifications
POST	/webhooks/stripe	Stripe billing events
POST	/webhooks/twilio/*	Twilio voice callbacks
GET	/webhooks/health	Webhook health check

WebSocket Endpoints

Path	Protocol	Description
/agent/ws	WSS	Agent hub (token auth)
/media-stream/	WSS	Twilio voice media stream

Deployment

Requirements

- Docker + Docker Compose
- PostgreSQL 15+
- Redis 7+
- Node.js 20+ (if running without Docker)

Docker Compose (Production)

```
# docker-compose.prod.yml
services:
  postgres:
    image: postgres:15-alpine
    volumes: [postgres_data:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres}"]

  redis:
    image: redis:7-alpine
    volumes: [redis_data:/data]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]

  backend:
    build: { context: ./mvp }
    environment:
      NODE_ENV: production
```

```

DATABASE_URL:
  postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
REDIS_URL: redis://redis:6379
ports: ["${BACKEND_PORT:-3000}:3000"]
depends_on: { postgres: { condition: service_healthy }, redis: { condition:
  service_healthy } }
healthcheck:
  test: ["CMD-SHELL", "wget -qO- http://localhost:3000/health || exit 1"]

frontend:
  build: { context: ./frontend, args: { BACKEND_URL: http://backend:3000 } }
  ports: ["${FRONTEND_PORT:-3001}:3001"]
  depends_on: { backend: { condition: service_healthy } }
```

Environment Variables

Required:

Variable	Description
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis connection string (default: redis://localhost:6379)
JWT_SECRET	32+ character secret for JWT signing

AI Models (at least one required):

Variable	Description
GEMINI_API_KEY	Google Gemini API key (BA Agent, primary generator)
OPENAI_API_KEY	OpenAI API key (GPT-o1 critique, Whisper transcription, voice bot)
ANTHROPIC_API_KEY	Reserved for server-side Claude integration

Optional Integrations:

Variable	Description
JIRA_API_TOKEN, JIRA_EMAIL, JIRA_BASE_URL	Jira ticket creation
GCHAT_WEBHOOK_URL	Google Chat notifications
GOOGLE_SERVICE_ACCOUNT_KEY, GOOGLE_DRIVE_FOLDER_ID	Google Docs PRD export
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	Google Meet OAuth (per-user)
GITHUB_TOKEN, GITHUB_DEFAULT_OWNER, GITHUB_DEFAULT_REPO	GitHub integration
SLACK_BOT_TOKEN	Slack notifications

Variable	Description
SENDGRID_API_KEY, EMAIL_FROM	Email notifications
TEAMS_WEBHOOK_URL	Microsoft Teams notifications
TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE_NUMBER	Voice + WhatsApp + SMS
AWS_S3_BUCKET, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY	Meeting audio storage
STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET	Billing
WORKOS_API_KEY, WORKOS_CLIENT_ID	Enterprise SSO
WEBHOOK_BASE_URL	Base URL for incoming webhooks (ngrok in dev)
OTEL_EXPORTER_OTLP_ENDPOINT, OTEL_SERVICE_NAME	OpenTelemetry tracing

Redis HA (Sentinel):

Variable	Description
REDIS_SENTINEL_HOSTS	Comma-separated host:port pairs
REDIS_SENTINEL_MASTER	Master name (default: mymaster)
REDIS_PASSWORD	Auth password
REDIS_DB	Database index (0-15, default: 0)

Async Job Processing

BullMQ processes 10 job types with separate workers:

Job Type	Trigger	Processing
MEETING_PROCESS	Meeting status -> completed	Extract transcript, update metadata
BA_AGENT_PROCESS	Manual or auto after meeting	Generate PRD via Gemini + AI Council
JIRA_SYNC	PRD approved + tickets created	Push tickets to Jira API
NOTIFICATION	Various events	Route to configured channel
DEV_AGENT_PROCESS	PRD approved	Dispatch to AgentHub for implementation
QA_AGENT_PROCESS	Implementation complete	Dispatch for code review
CLARIFICATION_TIMEOUT	Periodic (every 1 hour)	Expire stale clarification requests


```
});  
// Routes to member's preferredChannel, logs in MessageLog table
```

Testing

Backend test suite uses Vitest. Frontend uses Vitest for unit tests and Playwright for E2E.

```
# Backend  
cd mvp  
npm test           # Vitest run  
npm run smoke-test # Quick health check  
  
# Frontend  
cd frontend  
npm test           # Vitest run  
npm run test:e2e   # Playwright  
npm run test:e2e:ui # Playwright interactive UI  
  
# Agent  
cd agent  
npm test           # Vitest run
```

Test philosophy from AGENTS.md: - TDD workflow: RED (failing test) -> GREEN (minimal implementation) -> REFACTOR - Unit tests for services, integration tests for routes, E2E for critical flows - 80%+ coverage target - Never fix tests to match broken code – fix the code

Integration Points

Jira

- **Direction:** One-way (PRD -> Jira tickets)
- **Mechanism:** REST API via `JiraService`, triggered by `JIRA_SYNC` BullMQ job
- **Ticket types:** Story, Task, Bug, Epic with story points, acceptance criteria, labels
- **Webhook:** Incoming `/webhooks/jira` for status sync back to `JiraTicket.externalKey`
- **Configuration:** API token + email + base URL via Settings UI

GitHub

- **Direction:** Bidirectional
- **Outbound:** PR creation, branch management via `GitHubService` (PAT or GitHub App token)
- **Inbound:** `/webhooks/github` for PR events (merge, review)
- **Used by:** Dev Agent (implementation PRs) and QA Agent (review feedback)

Google Meet

- **OAuth:** Per-user OAuth flow via GoogleAuthService
- **Flow:** User authorizes -> Drive webhook watches for new recordings -> auto-extract transcript
- **Token storage:** AES-256-GCM encrypted in GoogleAuthToken table

Google Docs

- **Direction:** One-way export (PRD -> Google Doc)
- **Mechanism:** Service account credentials, writes to configured Drive folder
- **Stored:** PRD.googleDocId and PRD.googleDocUrl

Twilio (Voice)

- **Dial-in bot:** Users call a phone number, Twilio streams audio via WebSocket (/media-stream/), processed by OpenAI Realtime API
- **Voice data:** Requirements, decisions, action items captured in meeting metadata
- **Channels:** Also powers WhatsApp and SMS via communication router

Stripe (Billing)

- **Subscription tiers:** free_trial, starter, team, business, enterprise
- **Usage tracking:** meetingsUsedThisMonth, prdsUsedThisMonth on Tenant
- **Webhooks:** /webhooks/stripe for subscription lifecycle events

Roadmap

Timeline	Feature	Status
Now	BA Agent + PRD pipeline	Validated
Now	WebSocket Agent Hub + Dev Agent	Validated
Now	Multi-channel communication	Implemented
Sprint 2	Email digest, advanced voice features	In progress
Sprint 3	Zoom/Teams connectors, ROI dashboard	Planned
Series A	Industry templates, managed VMs, native installers	Planned

Architecture Decisions

Why Fastify (not Express)?

Fastify's plugin system provides natural dependency injection via `decorate()`. The schema-based validation (integrated with Zod) catches malformed requests before they hit business logic. Fastify 5 benchmarks at ~2x Express throughput with lower memory overhead.

Why Prisma 7 with pg adapter?

Prisma 7's adapter pattern (`@prisma/adapter-pg`) allows using a raw `pg.Pool` underneath, which enables connection pooling and direct SQL when needed. The `$extends` API powers the RLS middleware without requiring database-level row policies.

Why separate Redis connections?

BullMQ requires `maxRetriesPerRequest: null` on its Redis connection to handle blocking operations. Using the same connection for caching would break general Redis operations. The codebase creates two connections: one for caching (SkillLoader, MemoryService) and one for BullMQ workers.

Why client-side Dev Agent?

Running code generation on the user's machine solves three problems: (1) no need to store repo credentials on Workforce0 servers, (2) users bring their own Claude subscription, and (3) Docker isolation ensures the agent cannot access anything beyond mounted volumes. The tradeoff is operational complexity for the user (Docker setup), mitigated by the one-line Docker run command.

Built with TypeScript. Multi-tenant. Multi-model AI. WebSocket agent architecture. Self-improving skill system.